2019 5th International Conference on New Media Studies
Bali, Indonesia | October 09-11, 2019

LSB Steganography and AES Encryption for Multiple PDF Documents

Naldiyanto Sofian
Department of Informatics
Universitas Multimedia Nusantara
Tangerang, 15810, Indonesia
naldiyanto.sofian@student.umn.ac.id

Arya Wicaksana
Department of Informatics
Universitas Multimedia Nusantara
Tangerang, 15810, Indonesia
arya.wicaksana@umn.ac.id

Seng Hansun
Department of Informatics
Universitas Multimedia Nusantara
Tangerang, 15810, Indonesia
hansun@umn.ac.id

***Abstract*—Security is one important issue regarding data and information today. Steganography along with cryptography is used to hide and secure information. It is already an established practice to put watermark and to secure a single document using encryption. However, for large information requiring multiple documents has not received any attention. Here a program which can do steganography to multiple PDF documents to increase its steganography capacity is proposed, it is named MPDFStego. One of the steganography technique chosen in this research is the Least Significant Bit (LSB). LSB technique can be applied to Portable Document Format (PDF) by making use of its Tj Operator. Secret text is encrypted using Advanced Encryption Standard with 128-bit key size in Cipher Block Chaining (CBC) mode before it is embedded into the PDF document(s). Recompression and decompression of the PDF document(s) are done outside MPDFStego, with the help of QPDF tool. Testing and evaluation show that the MPDFStego is capable of hiding the secret text into one or many PDF documents (s).**

Keywords— AES, LSB, PDF, Steganography, Tj Operator

I. Introduction

Security is one of many important factors in communication in the world now [1]. One way to hide information is by doing steganography [2]. Steganography is used so that the message which is sent is not easily found by the unintended receiver [3]. It is different from encrypting information. Encryption changes the form of information into something that cannot be understood (called ciphertext), but the existence of the information is not hidden [3].

Least Significant Bit (LSB) substitution or overwriting is one of the common steganography technique to hide information in audio or image [4]. LSB technique can also be applied to Portable Document Format (PDF) file by making use of its existing operators' value, such as Tc, Tw, and Tj Operator [2].

Reference [2] states that the LSB technique is considered better than other PDF steganography technique, such as between-word/between-character embedding by [5], and incremental updates by [6]. These techniques create new components which are invisible in PDF file and cannot be seen if it is opened with a PDF reader program. But it still can be seen in their source code [2]. On the other hand, the LSB technique only makes use of components which are already existed, Tj Operators, and does not create any new components [2].

Encryption can be done before the information is hidden as an additional layer of security [7]. Encryption of information can be done using the Advanced Encryption Standard (AES). AES algorithm with 128-bit key size and Cipher Block Chaining (CBC) mode can be used to encrypt the information before it is hidden.

PDF files steganography has not received much attention compared to other technique, such as image steganography [2]. This is due to the fact that PDF files have low capacity and text-based file format makes it hard to hide data into [2]. Therefore, a program which can do LSB steganography to multiple PDF documents is built. Information is encrypted with AES-128 CBC prior to hiding.

II. Theories

A. Least Significant Bit

One of the spatial domain technique which is easy to implement is Least Significant Bit (LSB) [8]. LSB means the least significant bit in a byte value of a pixel in an image [8]. LSB-based image steganography attaches the hidden information in the least significant bit of a pixel of a cover image [8]. This concept exploits the precision that technology is capable of, higher than what human eye can interpret [8]. Hence, small changes that affect the image (usually color) will hardly differ from the original image if it is only seen by human eye [8].

Reference [4] gives an example of LSB steganography. Consider the following array of bytes of bits [4].

10010101 00001101 11001001 10010110

00001111 11001011 10011111 00010000

If "G" (with ASCII of 71 and bits of 01000111) is going to be hidden inside, then the bits will be changed into the following [4].

10010100 00001101 11001000 10010110

00001110 11001011 10011111 00010001

From the array above, it is seen that the least significant bits of each byte are the bits which represent "G" [4].

B. Advanced Encryption Standard

Advanced Encryption Standard is a crypthographic algorithm, made to replace Data Encryption Standard (DES) [9]. AES algorithm uses cryptographic keys with size of 128-, 192-, or 256-bit to encrypt or decrypt data in form of block with size of 128-bit [10].

AES consists of four different sub-processes [9]. One of the sub-process, ShiftRows, is a simple permutation [9]. The other three sub-processes, which are Substitute Bytes, MixColumns, and AddRoundKey, are substitution processes [9].

978-1-7281-0726-4/19/$31.00 ©2019 IEEE

Authorized licensed use limited to: Jain University. Downloaded on March 26,2026 at 09:42:11 UTC from IEEE Xplore. Restrictions apply.

The number of rounds in AES is vary, depends on the key size used [11]. AES with 128-bit key size uses 10 rounds, 192-bit key uses 12 rounds, and 256-bit key uses 14 rounds [11]. Each round utilizes different 128-bit round key, which is come from initial AES key [11].

Before the first round begins, AddRoundKey is done with the initial key and the plaintext to be hidden [9] [11]. This process XOR the plaintext with the key [9] [11]. The product of this process is used as input to the first round [9] [11].

Reference [9] states that each round of AES encryption has four sub-processes as the following.

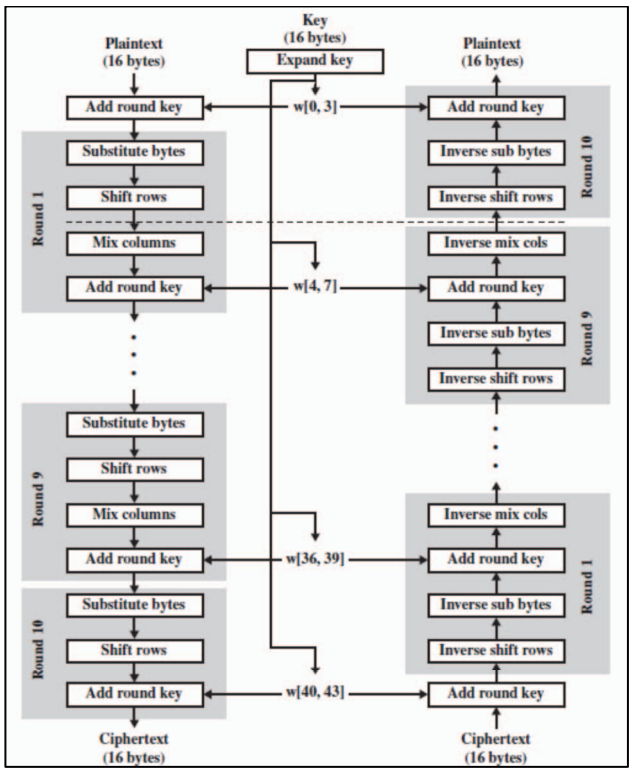

Fig. 1. Encryption and Decryption of AES [9]

a. Substitute Bytes

16 input bytes which are arranged in form of a matrix will be substituted using a fixed table (S-box) which is come from AES design [9] [11]. The product is a matrix which consists of four rows and four columns [9] [11]. For example, a hexadecimal of 19 will be substituted to D4 using the S-box [9] [11].

		y															
		0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
x	0	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
	1	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
	2	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
	3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
	4	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
	5	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
	6	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
	7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
	8	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
	9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
	A	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
	B	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
	C	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
	D	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
	E	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
	F	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

Fig. 2. Substitution Box (S-Box) for AES Encryption [9]

b. ShiftRows

Each row of the input matrix will be shifted to the left [9]. All values which are out of bound will be inserted back from the right of each row [9]. The following are the steps needed to do ShiftRows [9].

- First row is not shifted.
- Second row is shifted one (byte) to the left.
- Third row is shifted twice to the left.
- Fourth row is shifted thrice to the left.

The product is a new matrix consists of 16 same bytes as input bytes, but with different position [9].

c. MixColumns

This process operates the matrix per column [11]. Basically, it multiplies two 8-bit numbers, then takes the remainder from modulo 283 using the binary XOR long division as the result [11].

Reference [11] states that matrix multiplication is done to each of the column in the input matrix.

$$\begin{bmatrix} b_{0,c} \\ b_{1,c} \\ b_{2,c} \\ b_{3,c} \end{bmatrix} = \begin{bmatrix} 2 & 3 & 1 & 1 \\ 1 & 2 & 3 & 1 \\ 1 & 1 & 2 & 3 \\ 3 & 1 & 1 & 2 \end{bmatrix} \otimes \begin{bmatrix} a_{0,c} \\ a_{1,c} \\ a_{2,c} \\ a_{3,c} \end{bmatrix}$$

Fig. 3. Matrix Multiplication in AES MixColumns in decimal [11]

$a_{i,c}$ is the input column, $b_{i,c}$ is the product column of the matrix multiplication process, and ⊗ operator means that matrix multiplication is done with • operator and XOR rather than using usual addition [11]. b column will replace a column as the product [11].

MixColumns is not performed at the last round [9] [11]. In AES-128, MixColumns is not performed at the 10th round [9] [11].

d. AddRoundKey

16 bytes in the matrix will be considered as 128 bits and XOR-ed with the 128 bits round key [9]. If this is performed in the last round, then the product is the ciphertext [9]. If not, the 128 bits product will be considered as a new input and the next round begins [9].

C. *Cipher Block Chaining*

Cipher-block Chaining (CBC) is an encryption mode which XOR every plaintext block with the previous ciphertext block [9]. The product will then be encrypted using the corresponding algorithm [9]. each of the next ciphertext block depends on the previous block [9]. The first plaintext block will be XOR-ed with a random initialization vector (IV) [9]. CBC mode needs padding to ensure that the last plaintext block is a full block (not partial) [9].

Authorized licensed use limited to: Jain University. Downloaded on March 26,2026 at 09:42:11 UTC from IEEE Xplore. Restrictions apply.

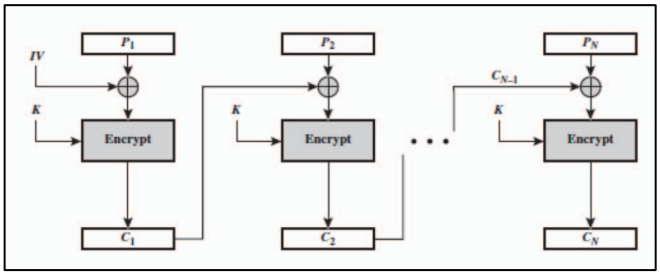

Fig. 4. Encryption in CBC Mode [9]

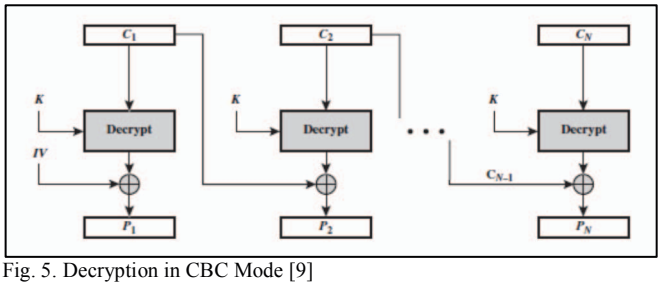

Fig. 5. Decryption in CBC Mode [9]

D. *javax.crypto*

javax.crypto is a package in Java library which provides classes and interfaces for cryptographic operation [12]. The following are some of the classes and interfaces which are used for AES-128 encryption.

- *javax.crypto.KeyGenerator*
- *javax.crypto.SecretKey*
- *javax.crypto.Cipher*
- *javax.crypto.spec.IvParameterSpec*
- *javax.crypto.spec.SecretKeySpec*

E. *PDF*

PDF document contains a collection of objects which determine how the contents are displayed [2]. One of the frequently-used object is stream object [2]. For example, text is contained inside a stream object [2]. There are other objects, such as numbers, strings, arrays, and dictionaries [2].

PDF file is usually compressed to save memory [2]. The source code of a PDF file can be read by decompressing the file [2]. Decompression of PDF file can be done with tools such as pdftk or QPDF [2]. Decompression process of a PDF file does not take a long time [2].

One of the operator to display text strings in PDF file is Tj Operator [2]. Tj Operator consists of an array of strings and numbers which represents characters and their spacing value [2]. For each Tj value, the current text position will be shifted by subtracting the value of that position [2]. If the next Tj value is negative, then the next character will be shifted further from the previous character (resulting in more space) [2]. If the next Tj value is positive, then the next character will be shifted closer to the previous character (resulting in less space) [2]. Tj Operators are often used to define variable space length between characters in justify-aligned text [2].

[(A)35(w)35(a)200(y)] TJ	Away
[(A)35(w)35(a)-200(y)] TJ	Awa y

Fig. 6. An Example of Tj Operator

Tj Block can be found using regular expression "\[(.*)\][ ]?TJ" [2]. After the Tj Block is found, Tj value can be obtained using regular expression "[>)](-?[0-9]+)[<(]" within the Tj Block [2].

III. Implementation and Testing

This part explains the result of the implementation and testing of LSB steganography and AES-128 encryption library to multiple PDF documents. A program is built and named MPDFStego for this purpose.

There are two tests which is performed on MPDFStego. One is to validate the implementation of AES-128 encryption. The other is to validate that MPDFStego is capable of doing LSB steganography to multiple PDF documents.

1. AES-128 Encryption Test

The implementation of AES-128 encryption is tested by comparing the ciphertext between MPDFStego's and an online tool which is based on the same input. The following is one of the AES-128 encryption test performed as described above.

- Hiding process will be done by MPDFStego as in Fig. 7.

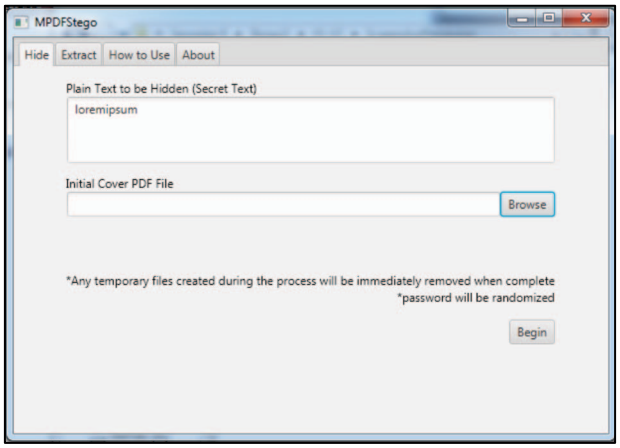

Fig. 7. Display of Hiding Tab Prior to Hiding Process

- Hiding process will generate 16 random characters as password. When hiding is finished, MPDFStego will generate a log contains information including password, ciphertext, and IV. Fig. 8 represents the log which is generated by the hiding process of MPDFStego.

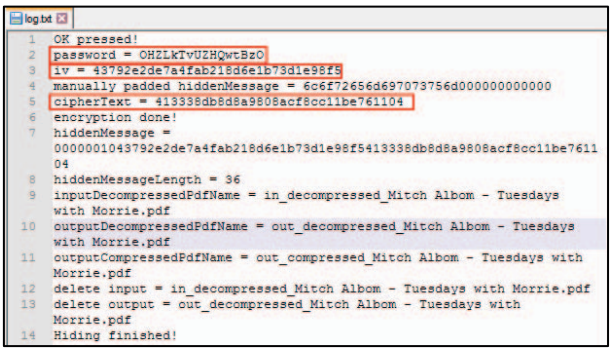

Fig. 8. MPDFStego Hiding Log

Authorized licensed use limited to: Jain University. Downloaded on March 26,2026 at 09:42:11 UTC from IEEE Xplore. Restrictions apply.

- The same password (key), IV, and plaintext is used as inputs to the online tool at http://aes.online-domain-tools.com/ as can be seen in Fig. 9.

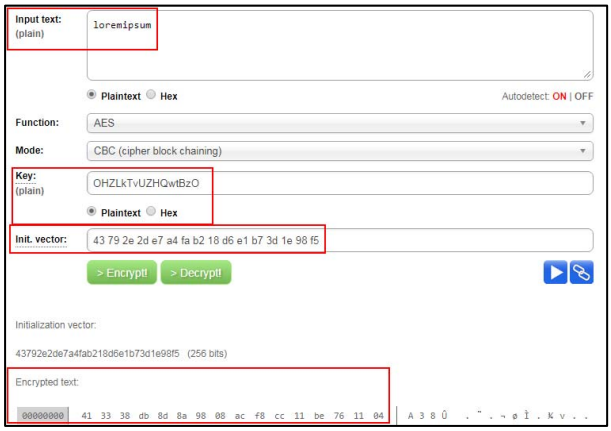

Fig. 9. Online Encryption Result

- Decryption of the ciphertext (Encrypted text) in the online tool produces the same plaintext as can be seen in Fig. 10.

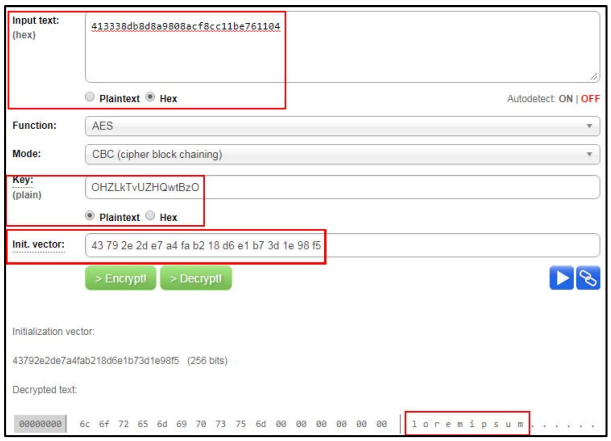

Fig. 10. Online Decryption Result

- The same plaintext is also obtained when the stego PDF file is extracted using MPDFStego with the same IV and password as the input. The extracted plaintext can be seen in Fig. 11.

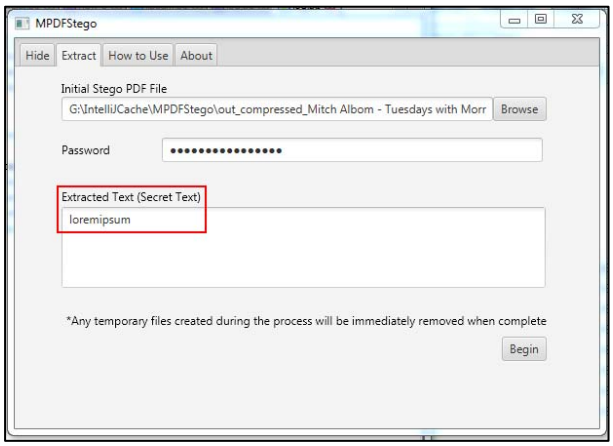

Fig. 11. Extraction of Secret Text with MPDFStego

- The plaintext can also be seen in the log after the extraction process has finished as can be seen in Fig. 12.

```
OK pressed!
inputDecompressedPdfName = in_decompressed_out_compressed_Mitch Albom - Tuesdays
with Morrie.pdf
cipherText.length() = 16
iv = 43792e2de7a4fab218d6e1b73d1e98f5
extractedMessageBytes = 413338db8d8a9808acf8cc11be761104
iv length = 16
extractedMessageBytes length = 16
decryptedText = loremipsum
```

Fig. 12. MPDFStego Extraction Log

The test shows that ciphertext from MPDFStego is the same as ciphertext from the online tool. This implies that the AES-128 library encryption is implemented correctly.

2. PDF LSB Steganography Tests

There are two tests which are performed regarding the LSB steganography. One is to validate the implementation of the LSB technique, and the other is to validate whether MPDFStego is capable of hiding data into multiple stego PDF files.

- Validating the LSB technique

The value of the Tj Operators are manually observed to see whether the LSB technique is correctly implemented. This test uses a special secret text "dua" (without quotation mark) to be hidden into a cover PDF file. Table 1 represents the bits needed to form the secret text "dua".

TABLE I. ASCII AND BINARY OF THE SECRET TEXT

Letter	ASCII (decimal)	Binary
d	100	01100100
u	117	01110101
a	97	01100001

The Tj Operator values before and after the hiding process are observed to be compared. Fig. 13 shows the Tj Operator values before the secret text is hidden in to it.

```
246  /GS8 gs
247  0 G
248  [(B)42(y)63( )-187(a)7(n)63( )-187(ou)63(t)28(l)91(i)91(v)63(e)7(d
     )-437(i)91(n)63(s)14(i)91(s)14(t)28(e)7(d )-374(pr)21(oc)7(u)63(r)21(e)7(d
     )-249(i)91(m)91(pr)21(ov)63(e)7(d )-374(a)7(m)91(. )-187(P)-6(a)7(i)91(d
     )-187(h)63(i)91(l)91(l)91( )-374(f)83(i)91(n)63(e)7( )-374(t)28(e)7(n)63(
     )-187(n)63(ow)35( )-187(l)91(ov)63(e)7( )-249(e)7(v)63(e)7(n)63(
     )-249(l)91(e)7(a)7(f)83(. )-249(S)-6(u)63(ppl)28(i)28(e)-55(d)-62( )] TJ
249  ET
250  Q
```

Fig. 13. Tj Operator Values prior to Hiding

Table 2 represents the expected Tj Operator values before and after the secret text is hidden. The very first cell represents the least significant bit of letter d, followed by its next bit (left of the previous bit), then followed by the least significant bit of letter 'u', and so on.

Authorized licensed use limited to: Jain University. Downloaded on March 26,2026 at 09:42:11 UTC from IEEE Xplore. Restrictions apply.

TABLE II. EXPECTED TJ OPERATOR'S VALUE BEFORE AND AFTER THE HIDING PROCESS

Letter (binary)	Tj Value Before (decimal)	Tj Value Before (binary)	Tj Value After (decimal)	Tj Value After (binary)
0	42	101010	42	101010
0	63	111111	62	11111**0**
1	-187	101000101	-187	101000101
0	7	111	6	11**0**
0	63	111111	62	11111**0**
1	-187	101000101	-187	101000101
1	63	111111	63	111111
0	28	11100	28	11100
1	91	1011011	91	1011011
0	91	1011011	90	101101**0**
1	63	111111	63	111111
0	7	111	6	11**0**
1	-437	1001001011	-437	1001001011
1	91	1011011	91	1011011
1	63	111111	63	111111
0	14	1110	14	1110
1	91	1011011	91	1011011
0	14	1110	14	1110
0	28	11100	28	11100
0	7	111	6	11**0**
0	-374	1010001010	-374	1010001010
1	21	10101	21	10101
1	7	111	7	111
0	63	111111	62	11111**0**

Fig. 14 below shows the Tj Operator values after secret text is hidden into the stego PDF file. Some of the values are changed to contain the bit of the secret text.

```
246  /GS8 gs
247  0 G
248  [(B)42(y)62( )-187(a)6(n)62( )-187(ou)63(t)28(l)91(i)90(v)63(e)6(d
     )-437(i)91(n)63(s)14(i)91(s)14(t)28(e)6(d )-374(pr)21(oc)7(u)62(r)21(e)7(d
     )-249(i)91(m)91(pr)21(ov)63(e)7(d )-374(a)7(m)91(. )-187(P)-6(a)7(i)91(d
     )-187(h)63(i)91(l)91(l)91( )-374(f)83(i)91(n)63(e)7( )-374(t)28(e)7(n)63(
     )-187(n)63(ow)35( )-187(l)91(ov)63(e)7( )-249(e)7(v)63(e)7(n)63(
     )-249(l)91(e)7(a)7(f)83(. )-249(S)-6(u)63(ppl)28(i)28(e)-55(d)-62( )] TJ
249  ET
250  Q
```

Fig. 14. Tj Operator's Value after Hiding

The test shows that the Tj Operator values in the Fig. 14 is the same as expected Tj Operator values after hiding in Table 2. This implies that LSB technique is implemented correctly.

- Validating the Hiding Process to Multiple PDF Files

In this test, MPDFStego is compared with another program, pdf_hide. The pdf_hide is a steganography program which hides a secret file into cover PDF file by changing the existing TJ Operators' value [13]. The aim is to prove that MPDFStego is capable of hiding secret text into multiple PDF files and extract the hidden secret text back.

There are five PDF documents which are used as cover files. The first four consists of one, two, three, and four lines, so their hiding capacity is limited. The last cover PDF file contains a very long passage to ensure the capacity is enough to contain all of the remainder of the secret text. Table 3 describes the input and output which are related to this test.

TABLE III. INPUT AND OUTPUT USED IN THE TEST

Input Text	Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut finibus nulla ac tellus efficitur aliquam. Praesent auctor hendrerit malesuada. In laoreet euismod feugiat. Vivamus imperdiet ex ac pharetra semper. Proin placerat enim vehicula, tincidunt purus et, malesuada diam. Fusce scelerisque vehicula felis quis ullamcorper. Morbi arcu mi, vehicula a purus et, maximus sagittis dui. Mauris sit amet purus sed tellus vulputate accumsan quis tincidunt diam. Suspendisse quis maximus turpis, nec rutrum nisi. Sed aliquam tempor metus, non pretium leo sodales vitae. Suspendisse potenti. Maecenas ut nisl porttitor, lobortis nulla nec, venenatis enim.
Input File (in order)	2Lines.pdf
	3Lines.pdf
	4Lines.pdf
	5Lines.pdf
	nLines.pdf
Output Password	EMXroYbcTPAZhCZx
Output File (in order)	out_compressed_2Lines.pdf
	out_compressed_3Lines.pdf
	out_compressed_4Lines.pdf
	out_compressed_5Lines.pdf
	out_compressed_nLines.pdf

On one hand, pdf_hide shows an error message when hiding the text (in a file) into 2Lines.pdf. It states that the hiding capacity of 2Lines.pdf is not enough to hide the message as can be seen in Fig. 15 below.

```
supherius@ubuntu:~$ cd pdf_hide/
supherius@ubuntu:~/pdf_hide$ python3 pdf_hide -o out_2Lines.pdf embed lipsum.txt
 2Lines.pdf
===================
This is PDF_HIDE v0.0
===================
Please enter key:
ERROR:  Not enough space available (only 28 available, 1336 needed)
supherius@ubuntu:~/pdf_hide$
```

Fig. 15. Error Message from pdf_hide while Hiding

On the other hand, MPDFStego prompts for a new cover PDF file if the previous cover file does not have enough hiding capacity to contain all of the secret text.

The extraction process of MPDFStego is done by inputting the corresponding stego PDF files into the program in order (2Lines.pdf as the initial stego PDF file, and nLines.pdf as the last stego PDF file). Password which is

Authorized licensed use limited to: Jain University. Downloaded on March 26,2026 at 09:42:11 UTC from IEEE Xplore. Restrictions apply.

given from the hiding process can be inputted in the password field as shown in Fig. 16.

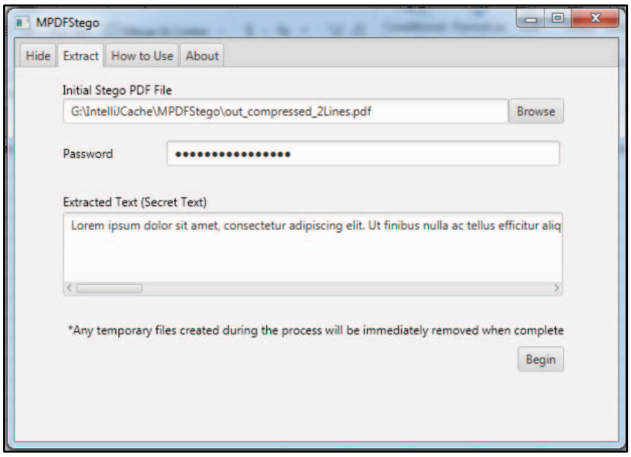

Fig. 16. Extraction Result of This Test

IV. Evaluation

Table 4 and Table 5 describes the stego PDF files which are produced both by MPDFStego and pdf_hide. The differences in file size can be found in the tables below.

TABLE IV. STEGO PDF FILES NLINES.PDF

Plain text	Plain text Size (bytes)	Cover PDF File Size (bytes)	Stego File Size (bytes)		Difference with Cover PDF File (bytes)	
			MPDF Stego	pdf_ hide	MPDF Stego	pdf_ hide
1	10	47.482	43.910	59.608	-3.572	12.126
2	43	47.482	44.010	59.627	-3.472	12.145
3	139	47.482	44.230	59.647	-3.252	12.165
4	243	47.482	44.461	59.613	-3.021	12.131

TABLE V. STEGO PDF FILES LATEXEXAMPLE.PDF

Plain text No.	Plain text size (Bytes)	PDF Cover file size (Bytes)	Stego file size (Bytes)		Size difference with PDF cover file (Bytes)	
			MPDF Stego	pdf_ hide	MPDF Stego	pdf_ hide
1	10	687.453	690.361	692.876	2.908	5.423
2	43		690.423	692.858	2.970	5.405
3	139		690.626	692.840	3.173	5.387
4	243		690.946	692.845	3.493	5.392

As can be seen above, stego PDF files from MPDFStego have smaller file size compared to pdf_hide. The increasing size of plaintext also increases the size of stego PDF file from MPDFStego, but it does not always increase the size of stego PDF file from pdf_hide.

V. Conclusion

Based on the tests which are performed, it is clear that LSB steganography and AES-128 encryption are implemented correctly in a program called MPDFStego. It is also proven that MPDFStego is capable of hiding the secret text into multiple PDF documents. The extraction of the secret text can be done by inputting the given password correctly and the corresponding stego PDF files in order.

Further study could be conducted on implementing this research on hardware. Recent studies on hardware simulation using virtual prototyping platform [14] and [15] enable early software development and system integration. The implementation of encryption and steganography in hardware could boost performances.

References

[1] K. M. Harshitha and P. A. Vijaya, "Secure Data Hiding Algorithm Using Encrypted Secret Message," *International Journal of Scientific and Research Publications,* vol. 2, no. 6, pp. 1-4, 2012.

[2] F. Alizadeh, N. Canceill, S. Dabkiewicz and D. Vandevenne, "Using Steganography to Hide Messages inside PDF Files," 30 December 2012. [Online]. Available: https://www.os3.nl/_media/2012-2013/courses/ssn/using_steganography_to_hide_messages_inside_pdf_files.pdf. [Accessed 4 June 2017].

[3] N. Meghanathan and L. Nayak, "Steganalysis Algorithms for Detecting the Hidden Information in Image, Audio, and Video Cover Media," *International Journal of Network Security & Its Application,* vol. 2, no. 1, pp. 43-55, 2010.

[4] G. C. Kessler, "An Overview of Steganography for the Computer Forensics Examiner," 2015. [Online]. Available: http://www.garykessler.net/library/fsc_stego.html. [Accessed 4 June 2017].

[5] I. S. Lee and W. H. Tsai, "A New Approach to Covert Communication via PDF Files," *Signal Processing,* pp. 557-565, 2010.

[6] H. Liu, L. Li, J. Li and J. Huang, "Three Novel Algorithms for Hiding Data in PDF Files Based on Incremental Updates," in *Digital-Forensics and Watermarking*, Atlantic City, 2011.

[7] S. Panghal, S. Kumar and N. Kumar, "Enhanced Security of Data using Image Steganography and AES Encryption Technique," *International Journal of Computer Applications,* pp. 1-4, 2016.

[8] B. Champakamala, K. Padmini and D. Radhika, "Least Significant Bit algorithm for image steganography," *International Journal of Advanced Computer Technology (IJACT),* vol. 3, no. 4, pp. 34-38, 2014.

[9] W. Stallings, Cryptography and Network Security Principles and Practice, Upper Saddle River, New Jersey: Pearson Education, Inc., 2014.

[10] Federal Information Processing Standards, "Announcing the ADVANCED ENCRYPTION STANDARD (AES)," 26 November 2001. [Online]. Available: http://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197.pdf. [Accessed 30 September 2017].

[11] C. Swenson, Modern Cryptanalysis Techniques for Advanced Code Breaking, Indiana: Wiley Publishing, Inc., 2008.

[12] Oracle, "javax.crypto (Java Platform SE 8)," [Online]. Available: https://docs.oracle.com/javase/8/docs/api/javax/crypto/package-summary.html. [Accessed 9 February 2018].

[13] N. Canceill, "Algorithm · ncanceill/pdf_hide Wiki · GitHub," 10 November 2013. [Online]. Available: https://github.com/ncanceill/pdf_hide/wiki/Algorithm.

[14] A. Wicaksana and C.M. Tang, "Virtual prototyping platform for multiprocessor system-on-chip hardware/software co-design and co-verification," Studies in Computational Intelligence (SCI), vol. 719, pp. 93-108, 2018.

[15] A. Wicaksana, C.M. Tang and M.S. Ng, "A scalable and configurable Multiprocessor System-on-Chip (MPSoC) virtual platform for hardware and software co-design and co-verification," CONMEDIA 2015 – International Conference on New Media 2015 (2016). DOI: 10.1109/CONMEDIA.2015.7449148

Authorized licensed use limited to: Jain University. Downloaded on March 26,2026 at 09:42:11 UTC from IEEE Xplore. Restrictions apply.